Uniform Cost Search

A practice workbook — worked trace, 11 exercises, full answer key

What UCS is

UCS explores a weighted graph in order of **cheapest cumulative cost from the start**, written $g(n)$. It is BFS with a priority queue in place of the FIFO queue — and it is Dijkstra's algorithm, run from a single source and stopped as soon as the goal is popped. Where BFS counts edges, UCS adds up weights.

Three rules that decide whether your trace is right:
1. Order the queue by $g(n)$, the cost so far. No heuristic — that would be A*.
2. Apply the goal test when a node is **popped**, never when it is generated.
3. All edge weights must be **non-negative**.

A fourth convention used throughout this workbook: never extend a path back onto a node it already contains. It cannot lose an optimal solution when weights are non-negative, and it keeps undirected traces readable.

	BFS	DFS	UCS
Queue order	FIFO (insertion)	LIFO (stack)	Lowest $g(n)$ first
Finds	Fewest edges	Some path	Cheapest total cost
Optimal?	Only if all weights equal	No	Yes, if weights ≥ 0
Complete?	Yes (finite b)	No (infinite depth)	Yes, if every step cost $\geq \varepsilon > 0$
Time	O(V + E)	O(V + E)	O(E log V) with a binary heap

Pseudocode

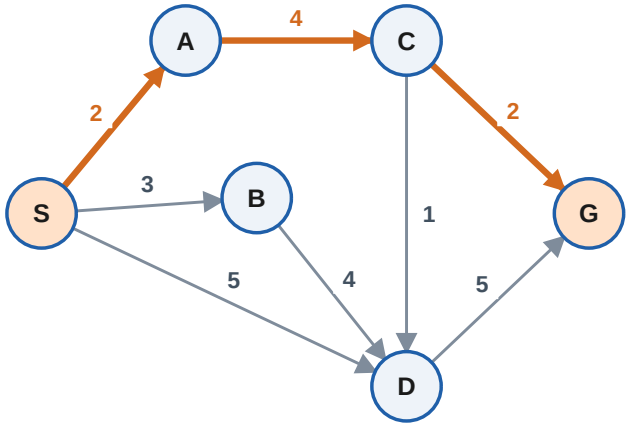
```
UCS(G, start, goal):
    frontier = priority queue ordered by g
    frontier.push(path=[start], g=0)
    expanded = empty set
    while frontier not empty:
        (path, g) = frontier.pop_min()
        node = last(path)
        if node == goal:              # test on POP
            return path, g
        if node in expanded:          # stale duplicate
            continue
        expanded.add(node)
        for (nbr, w) in neighbours(node):
            frontier.push(path + [nbr], g + w)
    return failure
```

Why the goal test waits for the pop. A goal node can be generated early along an expensive route while a cheaper route to the same goal is still sitting in the queue. Only when the goal reaches the front of the queue do you know nothing cheaper remains. Question 6 makes this concrete.

Stale entries. Pushing whole paths means a node can appear in the queue several times. Rather than searching the heap to update it, let the duplicates sit there and discard them on pop if the node was already expanded — this is *lazy deletion*, and it is why traces often show pops that expand nothing.

Worked example

Directed graph, start **S**, goal **G**. Successors taken in alphabetical order.



Full trace

Current (path & cost)	Newly added	Priority queue after
—	—	[S](0)
[S] (0)	[S,A](2) [S,B](3) [S,D](5)	[S,A](2) [S,B](3) [S,D](5)
[S,A] (2)	[S,A,C](6)	[S,B](3) [S,D](5) [S,A,C](6)
[S,B] (3)	[S,B,D](7)	[S,D](5) [S,A,C](6) [S,B,D](7)
[S,D] (5)	[S,D,G](10)	[S,A,C](6) [S,B,D](7) [S,D,G](10)
[S,A,C] (6)	[S,A,C,D](7) [S,A,C,G](8)	[S,A,C,D](7) [S,B,D](7) [S,A,C,G](8) [S,D,G](10)
[S,B,D] (7)	skip — D already expanded	[S,A,C,D](7) [S,A,C,G](8) [S,D,G](10)
[S,A,C,D] (7)	skip — D already expanded	[S,A,C,G](8) [S,D,G](10)
[S,A,C,G] (8) ✓	GOAL FOUND!	

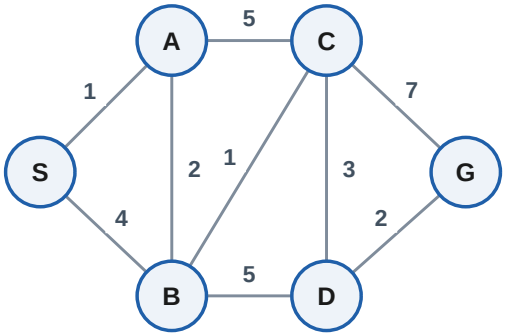
Result: optimal path S → A → C → G at total cost **8**.

Notice rows 7 and 8. Both [S,B,D](7) and [S,A,C,D](7) are cheaper than the goal entry [S,A,C,G](8), so UCS pops them first — but D was already expanded at cost 5, so both are discarded without generating anything. Many textbook tables omit these dead pops, which makes the trace look as though it jumps from 6 straight to 8.

Exercises

The answer key starts on page 7. Successors in alphabetical order throughout.

Question 1 — Trace UCS



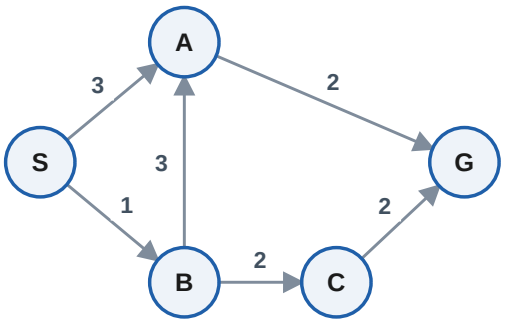
Graph **G1** (undirected). Start **S**, goal **G**.

(a) Trace UCS: give the order in which nodes are expanded.
(b) What is the optimal path and its cost?
(c) How many entries were still sitting in the queue when the goal was popped?

Question 2 — BFS would get this wrong

Still on **G1**. (a) What path does plain BFS return from S to G, and what does that path actually cost? (b) By how much does BFS overshoot the optimum? (c) State the exact condition under which BFS and UCS always agree.

Question 3 — Ties



Graph **G2** (directed). Start **S**, goal **G**.

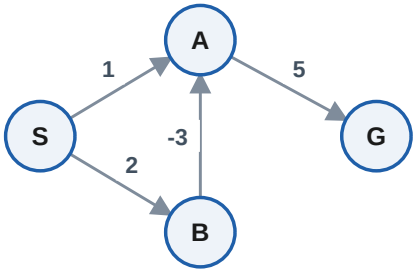
(a) Find every optimal path, not just one.
(b) Which one does UCS return under alphabetical tie-breaking, and why is that choice arbitrary rather than principled?

Question 4 — Complete the trace table

Fill in the UCS trace for **G1** (Question 1) from S to G. Include the pops that get discarded as stale — they are part of the algorithm's real behaviour.

Step	Popped (path & cost)	Newly added	Priority queue after
1			
2			
3			
4			
5			
6			
7			
8			
9			

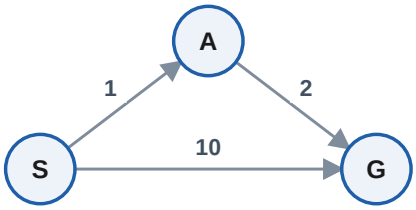
Question 5 — A negative edge



Graph **G3** has one negative edge, B→A of weight −3. Start **S**, goal **G**.

(a) Trace UCS. What path and cost does it report?
(b) What is the true cheapest path?
(c) Explain precisely which step of the algorithm the negative edge invalidates. Name an algorithm that would handle it.

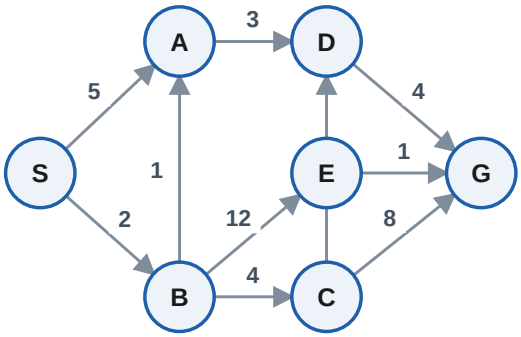
Question 6 — Goal test on pop, not on push



Graph **G4**. Start **S**, goal **G**.

(a) Suppose you (wrongly) return as soon as a goal node is *generated*. What answer do you get?
(b) What does correct UCS return?
(c) Write the smallest graph you can that exposes this bug.

Question 7 — Counting expansions



Graph **G5** (directed). Start **S**, goal **G**.

(a) List the nodes UCS expands, in order.
(b) Give the optimal path and cost.
(c) Which nodes does UCS never expand at all, and what does that tell you about the work it avoided?

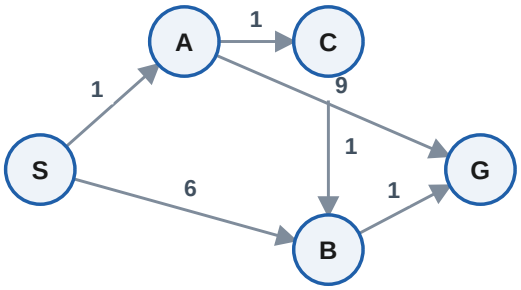
Question 8 — UCS on a weighted grid

S	1	1	1
9	9	9	1
1	1	1	1
E	9	9	1

Each number is the cost to **enter** that cell. Move up / down / left / right only. Entering **E** costs 1; leaving **S** costs nothing.

(a) What is the cheapest total cost from S to E?
(b) Give the cheapest route.
(c) The fewest-move route is just 3 moves, straight down the left column. What does it cost, and why does UCS reject it?

Question 9 — Three algorithms, one graph



Graph **G6** (directed). Start **S**, goal **G**. For each of BFS, DFS and UCS give the path returned and its cost. Then say which algorithm you would choose here and why.

Question 10 — Short answer

(a) Under what condition does UCS behave exactly like BFS?

(b) UCS and Dijkstra are usually described as the same algorithm. Name one real difference in how they are typically stated.

(c) Why is UCS complete only if every step cost is at least some $\varepsilon > 0$?

(d) Storing whole paths in the queue (as this workbook does) is memory-hungry. What do real implementations store instead, and how do they recover the path?

Question 11 — Design a graph

Construct a directed graph with at most 6 nodes in which UCS expands *every* node before it finds the goal. Then modify one edge weight so that UCS expands only two nodes. Explain what property of the graph you changed.

Answer Key

Question 1

(a) Expansion order: **S** → **A** → **B** → **C** → **D**

(b) Optimal path **S** → **A** → **B** → **C** → **D** → **G**, cost **9**.

(c) See the completed table in Question 4 — the queue still held several entries, all with cost ≥ 9. UCS stops the moment the goal reaches the front; it never needs to empty the queue.

Question 2

(a) BFS returns **S** → **A** → **C** → **G**, which costs **13** — it is the fewest-edge route, not the cheapest one.

(b) BFS overshoots by **4** (13 against the optimal 9).

(c) They agree whenever every edge has the **same** weight. Then cost is a fixed multiple of depth, so ordering by cost and ordering by depth are the same ordering.

Question 3

(a) Two optimal paths, both cost **5**: **S** → **A** → **G** and **S** → **B** → **C** → **G**.

(b) UCS returns **S** → **A** → **G**. Which of the two surfaces first depends entirely on how the priority queue breaks ties — push order, heap layout, or an explicit tiebreaker. It is an implementation artefact, not a property of the graph, which is why exam questions always fix a tie-breaking rule before asking for 'the' answer.

Question 4

Completed UCS trace for G1 from S to G:

Current (path & cost)	Newly added	Priority queue after
—	—	[S](0)
[S] (0)	[S,A](1) [S,B](4)	[S,A](1) [S,B](4)
[S,A] (1)	[S,A,B](3) [S,A,C](6)	[S,A,B](3) [S,B](4) [S,A,C](6)
[S,A,B] (3)	[S,A,B,C](4) [S,A,B,D](8)	[S,A,B,C](4) [S,B](4) [S,A,C](6) [S,A,B,D](8)
[S,B] (4)	skip — B already expanded	[S,A,B,C](4) [S,A,C](6) [S,A,B,D](8)
[S,A,B,C] (4)	[S,A,B,C,D](7) [S,A,B,C,G](11)	[S,A,C](6) [S,A,B,C,D](7) [S,A,B,D](8) [S,A,B,C,G](11)
[S,A,C] (6)	skip — C already expanded	[S,A,B,C,D](7) [S,A,B,D](8) [S,A,B,C,G](11)
[S,A,B,C,D] (7)	[S,A,B,C,D,G](9)	[S,A,B,D](8) [S,A,B,C,D,G](9) [S,A,B,C,G](11)
[S,A,B,D] (8)	skip — D already expanded	[S,A,B,C,D,G](9) [S,A,B,C,G](11)
[S,A,B,C,D,G] (9) ✓	GOAL FOUND!	

Question 5

(a) UCS reports **S** → **A** → **G** at cost **6**.

(b) The true cheapest path is **S** → **B** → **A** → **G** at cost **4** — UCS is wrong by 2.

(c) UCS marks A as expanded when it pops A at g=1, on the assumption that no cheaper route to A can appear later. The negative edge B→A breaks exactly that assumption: popping B at g=2 yields A at g=−1, but A is already closed and the improvement is thrown away. Use **Bellman–Ford** (or Johnson's algorithm for all pairs) when negative weights are present; note that a negative *cycle* makes 'cheapest path' meaningless for any algorithm.

Question 6

(a) Testing on generation, you expand S, immediately produce [S,G](10), and return cost **10**.

(b) Correct UCS pops [S,A](1) first, generates [S,A,G](3), and returns **S** → **A** → **G** at cost **3**.

(c) Three nodes and three edges is the minimum: one expensive direct edge to the goal plus a cheaper two-edge detour. Any graph where the goal is reachable by both a costly short route and a cheap longer one exposes the bug.

Question 7

(a) Expansion order: **S** → **B** → **A** → **C** → **D**

(b) Optimal path **S** → **B** → **A** → **D** → **G**, cost **10**.

(c) Never expanded: **E**. (G is popped and returned, not expanded.) E sat in the queue at cost 14, above the goal's 10, so UCS could prove no route through E could beat the path it already had. That is the payoff of expanding in cost order — the search halts the instant the goal becomes cheapest, and the expensive region of the graph is never touched.

Question 8

(a) Cheapest total cost = **9**.

(b) Route (row, col), top-left is (0,0): (0,0) → (0,1) → (0,2) → (0,3) → (1,3) → (2,3) → (2,2) → (2,1) → (2,0) → (3,0) — 9 moves, right along the top, down the cheap right-hand column, then back left along row 2.

(c) Straight down the left column costs 9 + 1 + 1 = **11** in only 3 moves. UCS rejects it because that single 9-cost cell outweighs the whole nine-move detour. This is the sharpest version of the BFS/UCS distinction: BFS would seize the 3-move route immediately and report it as optimal, because BFS optimises *moves* while UCS optimises *cost*. Whenever a step's price varies, those two answers come apart.

Question 9

BFS: **S** → **A** → **G**, cost 10 (2 edges — the fewest).

DFS: **S** → **A** → **C** → **B** → **G**, cost 4.

UCS: **S** → **A** → **C** → **B** → **G**, cost **4** — the most edges of the three, and the cheapest.

Choose UCS whenever the edge weights mean something (distance, time, money, fuel). BFS only answers 'fewest hops', and DFS answers neither question reliably — it returns whichever path its tie-breaking rule happens to walk down first.

Question 10

(a) When every edge has identical weight. UCS then pops nodes in non-decreasing depth order, exactly as BFS does.

(b) They are the same search. The usual difference is framing: Dijkstra is normally stated as computing shortest distances from the source to **all** nodes, relaxing edges and updating a distance array; UCS is stated as a goal-directed search that **stops early** when the goal is popped, and is often written with lazy deletion instead of a decrease-key operation.

(c) With step costs allowed to approach zero, an infinite path can have finite total cost. UCS would keep expanding along that path forever, its g creeping upward but never exceeding the cost of the goal. A positive lower bound ε guarantees g grows without bound, so any finite-cost goal is reached in finitely many expansions.

(d) Store one `parent` pointer and one g value per node rather than a full path per queue entry. Once the goal is popped, walk the parent pointers backwards from the goal and reverse the result — O(V) memory instead of O(V) copies of paths of length up to V.

Question 11

One answer: a chain S→A→1 with cheap weights and a single expensive edge to the goal, e.g. S→A(1), A→B(1), B→C(1), C→D(1), D→G(1) plus S→G(100). Every node is cheaper than the direct goal edge, so UCS expands all of them before popping G.

Now change S→G from 100 to 1. UCS pops S, then G is tied with A at cost 1 and is popped essentially immediately — two expansions.

What changed is the **relationship between the goal's cost and the costs of everything else**. UCS expands precisely those nodes whose optimal g is less than the optimal cost of the goal. Lower the goal's cost below the rest of the graph and that set collapses to almost nothing; raise it above everything and the set becomes the whole graph.

Next step: take any graph here and add a heuristic $h(n)$ — a guess at the remaining cost to the goal. Ordering by $g(n) + h(n)$ instead of $g(n)$ turns UCS into **A***, and setting $h(n) = 0$ turns it straight back into UCS. Every trace-table skill here transfers unchanged.